# Linux 与 Git 及 C++进阶综合技术报告

## 一、实验简介

本次实验分为两大模块。第一模块为 Linux 与 Git 基础考核，包含 Linux 基础 7 关、Git 基础 6 关，合计 13 个关卡，覆盖 Linux 基础命令、Shell 脚本开发、后台进程管理、Git 版本控制。第二模块为 C++工程进阶练习，共 3 关，包含读数统计、设备巡检记录、固定容量读数窗口，训练数值统计、结构体业务处理、环形缓冲区内存管理。报告记录全部实操命令、通关代码、解题思路与踩坑总结。

## 二、关卡详细记录

### 第 1 关：找到入口

**目标**：熟悉 Linux 路径切换与目录浏览操作

```
Bash
cd /workspace
ls
```

心得：执行命令前确认当前工作目录，防止路径错误导致找不到文件。

---

### 第 2 关：两段信号的提取与拼接

**目标**：检索目标文件、提取密钥，按 A 在前 B 在后的顺序拼接生成 flag 文件

```
Bash
# 查找信号文件
find /workspace/ -type f | grep "signal-A"
find /workspace/ -type f | grep "signal-B"
# 提取密钥存入环境变量
export KEY=$(grep "^ACCESS_KEY=" /workspace/catalog/words.txt |
cut -d'=' -f2)
echo $KEY
```

```
# 处理 A 信号
cp /workspace/archive/2026/signal-A.bin /workspace/input
/workspace/exam -e "$KEY"
cp /workspace/output /workspace/flag
# 处理 B 信号，>>为追加写入，不会覆盖原有内容
cp /workspace/inbox/signal-B.bin /workspace/input
/workspace/exam -e "$KEY"
cat /workspace/output >> /workspace/flag
ls -lh /workspace/flag
```

心得：`>`是覆盖写入，`>>`是追加写入；本关卡核心就是避免第二次运行程序覆盖已有结果。

---

## 第 3 关：读日志并修复

**目标**：读取日志文件，过滤错误信息，定位程序异常问题

```
Bash
cat /workspace/log.txt
less /workspace/log.txt
grep -i error /workspace/log.txt
```

心得：使用 grep 筛选关键词，可以快速在大量日志中定位报错信息，提升排错效率。

---

## 第 4 关：让命令找到路 route.sh

**目标**：编写 Shell 脚本，完成参数校验，调用探测程序，根据程序返回码输出对应结果

```
Bash
#!/bin/bash
if [ -f "/workspace/job.env" ]; then
  source "/workspace/job.env"
fi
if [ -z "$ROUTE_RESULT" ]; then
  echo "usage"
  exit 64
fi
if [ $# -ne 2 ]; then
  echo "usage"
  exit 64
fi
```

```
MODE="$1"
ITEM="$2"
if [[ ! "$MODE" =~ ^(ok|retry|fatal)$ ]] || [[ ! "$ITEM" =~ ^[a-
zA-Z0-9]+$ ]];then
  echo "usage"
  exit 64
fi
export ACCESS_TOKEN
export TASK_TOOL_DIR
export PATH="$TASK_TOOL_DIR:$PATH"
OUTPUT=$(task-probe "$MODE" "$ITEM" 2>/dev/null)
RC=$?
case $RC in
0)
  printf "receipt:%s\n" "$ITEM" > "$ROUTE_RESULT"
  exit 0
;;
11)
  printf "retry:%s\n" "$ITEM" > "$ROUTE_RESULT"
  exit 75
;;
23)
  printf "rejected:%s\n" "$ITEM" > "$ROUTE_RESULT"
  exit 65
;;
*)
  exit $RC
;;
esac
```

心得：`[[ ]]`支持正则表达式；case 语句每个分支末尾必须写`;;`；`2>/dev/null` 用于屏蔽多余错误输出。

---

## 第 5 关：照看后台任务 manage.sh

**目标**：编写脚本实现后台进程的 start、status、report、stop 功能，完成进程状态检测与优雅关停

```
Bash
#!/bin/bash
if [ -z "$WORK_DIR" ];then
  echo "usage"
```

```
  exit 64
fi
PID_FILE="$WORK_DIR/worker.pid"
WORKER_BIN="/opt/recruitment/bin/slow-worker"
mkdir -p "$WORK_DIR"
is_worker_running(){
  [ ! -f "$PID_FILE" ] && return 1
  local pid=$(cat "$PID_FILE" 2>/dev/null)
  if ! kill -0 "$pid" 2>/dev/null; then return 1; fi
  if ps -p "$pid" -o stat= | grep -q "Z"; then return 1; fi
  if ! ps -p "$pid" -o cmd= | grep -q "slow-worker"; then return
1; fi
  return 0
}
do_start(){
  if is_worker_running; then exit 0; fi
  nohup "$WORKER_BIN" < /dev/null > /dev/null 2>&1 &
  echo $! > "$PID_FILE"
  exit 0
}
do_status(){
  if is_worker_running; then echo "running"; exit 0; else echo
"stopped"; exit 0; fi
}
do_report(){
  if ! is_worker_running; then exit 1; fi
  local pid=$(cat "$PID_FILE")
  local report_file="$WORK_DIR/resource.txt"
  echo "PID=$pid" > "$report_file"
  ps -p "$pid" -o pid,ppid,user,%cpu,%mem,stat,start,time,cmd >>
"$report_file"
  echo "===== TOP SNAPSHOT =====" >> "$report_file"
  timeout 0.5s top -b -n 1 -p "$pid" >> "$report_file" 2>/dev/null
  exit 0
}
do_stop(){
  echo "stopped" > "$WORK_DIR/stopped.log"
  if [ ! -f "$PID_FILE" ];then
    exit 0
  fi
  local pid=$(cat "$PID_FILE" 2>/dev/null)
  if [ -z "$pid" ];then
    rm -f "$PID_FILE"
    exit 0
```

```
  fi
  kill -TERM "$pid" 2>/dev/null
  kill -TERM -"$pid" 2>/dev/null
  local count=0
  while [ $count -lt 5 ]; do
    if ! kill -0 "$pid" 2>/dev/null; then break; fi
    if ps -p "$pid" -o stat= 2>/dev/null | grep -q "Z"; then
break; fi
    sleep 0.1
    count=$((count + 1))
  done
  if kill -0 "$pid" 2>/dev/null; then
    kill -9 "$pid" 2>/dev/null
    kill -9 -"$pid" 2>/dev/null
  fi
  rm -f "$PID_FILE"
  echo "stopped" > "$WORK_DIR/stopped.log"
  exit 0
}
case "$1" in
start) do_start ;;
status) do_status ;;
report) do_report ;;
stop) do_stop ;;
*) echo "usage"; exit 64 ;;
esac
```

心得：多重校验避免脏 PID、僵尸进程误判；优先使用 TERM 优雅终止进程，超时后再使用 kill- 9 强制结束；nohup 可以将程序放到后台持续运行。

---

## 第 6 关：交给服务账号

**目标**：理解 Linux 多用户机制，掌握用户切换，使用低权限账号运行服务保障安全

```
Bash
whoami
su 服务账号名
```

心得：生产环境尽量不使用 root 账号运行服务，采用低权限服务账号，降低安全风险。

---

## 第 7 关：装好工具并运行

**目标**：使用 apt 包管理器安装 Git，验证软件环境

```
Bash
apt update
apt install git -y
git --version
```

心得：安装软件前更新软件源，保证软件包可以正常下载安装。

---

## 第 8 关：Git 就在本地

**目标**：初始化本地 Git 版本仓库，指定 main 为主分支

```
Bash
cd /workspace/repo
git init -b main
```

心得：初始化本地 Git 仓库，并指定 main 作为主分支。

---

## 第 9 关：看清工作区和暂存区

**目标**：练习文本写入，区分工作区与暂存区，查看文本隐藏字符

```
Bash
cat > notes.txt << 'EOF'
local git practice
first local note
EOF
cat -A notes.txt
```

心得：区分工作区原始文件；cat- A 可以查看文本里隐藏换行等不可见字符。

---

## 第 10 关：留下第一个版本

**目标**：完成文件暂存与版本提交，生成第一条版本记录

```
Bash
git add notes.txt
```

```
git commit -m "learn: first local commit"
```

心得：标准提交流程，add 把文件加入暂存区，commit 生成版本记录。

---

## 第 11 关：找回历史中的内容

**目标**：查看提交历史，从历史版本恢复单个文件

```
Bash
git log
git checkout <commit-id> notes.txt
```

心得：git log 查看全部提交历史，支持从历史版本恢复单个文件。

---

## 第 12 关：让两条历史汇合

**目标**：创建分支，开发分支代码，完成分支合并，保留合并记录

```
Bash
git switch -c topic
echo "topic ready" > topic.txt
git commit -m "topic ready"
git switch main
echo "main ready" > main.txt
git commit -m "main ready"
git merge topic --no-ff -m "Merge topic into main"
```

心得：`--no- ff` 参数会保留分支合并记录，不会压缩提交历史。

---

## 第 13 关：完成可逆修改

**目标**：使用 revert 实现安全回退，不修改原有提交历史

```
Bash
git revert HEAD --no-edit
```

心得：git revert 会新增一条反向提交，不会修改已有提交历史，适合公共分支安全回退。

---

## 第 14 关：读数统计（C++基础修复）

**目标**：修复统计程序，读取一组整数，输出总数、总和、最小值、最大值

```C++
#include <iostream>
int main() {
    int n = 0;
    if (!(std::cin >> n)) return 1;
    long long sum = 0;
    int count = 0;
    int max_val = -2147483648;
    int min_val = 2147483647;
    for (int i = 0; i < n; ++i) {
        int num;
        std::cin >> num;
        sum += num;
        count++;
        if (num > max_val) max_val = num;
        if (num < min_val) min_val = num;
    }
    std::cout << "count=" << count << "\n";
    std::cout << "sum=" << sum << "\n";
    std::cout << "min=" << min_val << "\n";
    std::cout << "max=" << max_val << "\n";
    return 0;
}
```

心得：求极值的初始化要点：最大值初始设为 int 最小值，最小值初始设为 int 最大值；提交前执行 `rm main` 删除本地编译产物，避免评测平台异常报错。

---

## 第 15 关：设备巡检记录（复合数据与条件查询）

**目标**：使用结构体存储设备巡检记录，统计不同设备的记录数量、累计工时，并找出工时最多的设备；当多个设备总工时并列最高时，输出排序靠前的设备

```C++
#include <iostream>
#include <string>
#include <vector>
#include <map>
using namespace std;
struct Record {
```

```
    string type;
    string status;
    int hours = 0;
};
int main() {
    int count = 0;
    if (!(cin >> count)) return 1;
    map<string, int> type_counts;
    map<string, int> type_hours;
    vector<string> order = {"printer", "scanner", "router",
"server"};
    for (const string& t : order) {
        type_counts[t] = 0;
        type_hours[t] = 0;
    }
    for (int i = 0; i < count; ++i) {
        Record r;
        cin >> r.type >> r.status >> r.hours;
        type_counts[r.type]++;
        type_hours[r.type] += r.hours;
    }
    for (const string& t : order) {
        cout << t << " " << type_counts[t] << " " << type_hours[t]
<< "\n";
    }
    string busiest = "printer";
    int max_hours = type_hours["printer"];
    for (const string& t : order) {
        if (type_hours[t] > max_hours) {
            max_hours = type_hours[t];
            busiest = t;
        }
    }
    cout << "busiest " << busiest << "\n";
    return 0;
}
```

心得：`std::map` 适合做分组聚合统计；vector 固化设备顺序，搭配>严格大于判断，解决并列最大值优先输出前置项的业务需求；学会通过 `ls -la`、`cat samples/*.in` 查看样例输入输出来挖掘隐藏业务规则。

---

## 第 16 关：固定容量读数窗口（内存控制与环形队列）

**目标**：实现最多保存最近 8 个读数的滑动窗口，修复数组越界 bug，环形覆盖旧数据。输出 kept 有效数量、最老数据、最新数据、总和

```
C++
#include <iostream>
#include <algorithm>
int main() {
    const int CAPACITY = 8;
    int window[CAPACITY] = {0};
    int count = 0;
    if (!(std::cin >> count)) return 1;
    for (int i = 0; i < count; ++i) {
        int value = 0;
        if (!(std::cin >> value)) return 1;
        // 取余实现环形覆盖，下标永远在 0~7 之间
        window[i % CAPACITY] = value;
    }
    int kept = std::min(count, CAPACITY);
    long long sum = 0;
    for (int i = 0; i < kept; ++i) sum += window[i];
    // 计算最老数据真实物理下标
    int oldest_index = (count > CAPACITY) ? (count % CAPACITY) :
0;
    std::cout << "kept=" << kept << "\n";
    if (kept > 0) {
        std::cout << "oldest=" << window[oldest_index] << "\n";
        std::cout << "last=" << window[(count - 1) % CAPACITY] <<
"\n";
    } else {
        std::cout << "oldest=0\nlast=0\n";
    }
    std::cout << "sum=" << sum << "\n";
    return 0;
}
```

心得：取余运算%是环形数组的核心，它可以将持续增加的索引映射到固定大小的物理数组，防止数组越界踩踏野内存；实现固定大小滑动窗口，是嵌入式、信号采集场景非常经典的数据结构。

## 三、综合实验总结

本次实验一共 16 个关卡，分为 Linux&Git 基础、C++工程进阶两大模块。

1. **Linux 模块**：掌握目录切换、文件检索、日志排查、环境变量、Shell 脚本编写、

进程启停管控、apt 软件安装；学会区分覆盖写入与追加写入、正则校验、脏 PID 处理，完成工程脚本开发。

2. **Git 模块**：掌握仓库初始化、文件暂存提交、分支创建、分支合并、历史文件恢复、安全 revert 回退，理解版本控制规范。

3. **C++进阶模块**：从基础数值统计，到结构体处理业务数据，再到环形缓冲区内存管理；学习极值初始化、溢出防护、容器使用、数组边界防护，贴近嵌入式、信号采集真实工程场景。

4. **通用排错收获**：注意边界测试，包括空输入、大数量输入、并列条件；遇到陌生题目优先阅读样例输入输出；数组越界、脚本返回码、Git 历史修改都是高频易错点。

## 四、注意

除了第四个和第五关之外其他大致搞懂，这两关是纯 AI,详细心路历程见感悟报告。